SIMATS ENGINEERING

DSA0114 – Object-Oriented Programming with C++

ASSIGNMENT 3 – CO3

Co3: constructors, destructors and operator overloading to control object creation, destruction and operator behaviour – default, parameterised, overloaded, copy and dynamic constructors, destructors, defining and overloading operators, and the rules for operator overloading.

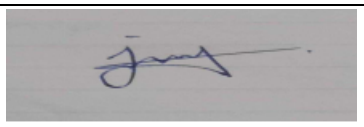
Student Name	Jannathul Firdaus. N	**Register Number**	192573025
Department / Section	B.Tech CSE - DS	**Academic Year / Semester**	2nd year
Faculty Name	S. Amirtha	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	13.11.2026	**Date of Evaluation**	
GitHub Repository	https://github.com/jannathulfirdaus17/DSA0114_192573025_Assignment3	**Compiler**	Code::Blocks IDE with the GNU GCC (G++) compiler.

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Constructors and Constructor Overloading	6		
2	Copy Constructors: Shallow and Deep Copies	7		
3	Dynamic Constructors and Destructors	7		
4	Overloading Arithmetic and Stream Operators	7		

5	Overloading Increment, Comparison and Subscript Operators	6		
6	The Rules of Operator Overloading	7		
7	Design a Complex-Number Class from First Principles	10		
	TOTAL	50		Percentage:

OVERALL REMARKS

		
Student Signature	**Faculty Signature**	**Program Director**

ASSIGNMENT 3 – CO3

CO3 Coverage: Construct C++ programs using constructors, destructors and operator overloading to control object creation, destruction and operator behaviour – default, parameterised, overloaded, copy and dynamic constructors, destructors, defining and overloading operators, and the rules for operator overloading.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 13.11.2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)
Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment3 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 - Wall -Wextra), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link:
https://github.com/jannathulfirdaus17/DSA0114_192573025_Assignment3

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assignment Principle

Every question asks you to design, implement, analyse and evaluate the solution yourself, so that you demonstrate evidence of:

Design → Implementation → Analysis → Evaluation

Q1 – Constructors and Constructor Overloading 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Library Book Records

A library creates Book objects in several situations: a new book with full details, a donated book with only a title, and an empty record that will be filled later. Some records were created with garbage values in the number of copies.

Implementation Tasks

You must:

• Write a default constructor, a parameterised constructor and overloaded constructors for the three situations

• Use a member initializer list in every constructor

• Use constructor delegation to avoid repeating code

• Show that no Book can be created with an invalid number of copies

Design & Analysis

You must:

• Explain when the compiler supplies a default constructor and when it does not

• Explain why member initializer lists are preferred to assignment inside the constructor body

• Explain the order in which members are initialised (declaration order, not list order) • Draw the state of three objects after construction

Experimental Investigation

You must:

• Create Book objects with every constructor and print them

• Remove the default constructor and record the compiler error when an array of Books is created

• Demonstrate a member initialised in the wrong order and the -Wreorder warning • Test a negative number of copies

Design Question

When should a constructor be marked explicit? Show a call that compiles without explicit and fails with it.

Alternative Concept and Complexity Analysis

If one constructor with default arguments (Book(std::string t = "", int copies = 0, …)) were used instead of several overloaded constructors and delegation, what would the solution be? Analyse the time and space of object creation (O(1) in both) and compare which design checks the number of copies in fewer places.

Q2 – Copy Constructors: Shallow and Deep Copies 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Hospital Prescription Records

A Prescription object holds a dynamically allocated array of medicine names. When a doctor copies a prescription to modify it for a follow-up visit, the original prescription also changes, and the program sometimes crashes when it ends.

Implementation Tasks

You must:

• Implement the class first without a copy constructor and observe the problem

• Implement a deep-copying copy constructor, a copy assignment operator and a destructor

• Verify that copies are independent

• Run the program under -fsanitize=address (or valgrind) for both versions Design & Analysis

You must:

• Draw memory diagrams for a shallow copy and a deep copy

• Explain why the shallow version crashes at the end (double delete) • State the Rule of Three and explain why the three functions go together • Explain the three situations in which the copy constructor is called

Experimental Investigation

You must:

• Add a message in the copy constructor and count how many copies are made when a Prescription is passed by value and by const reference

• Measure the time to copy prescriptions of 10, 1,000 and 100,000 medicines • Report the sanitizer output for both versions

• Test self-assignment (p = p)

Conclusion

When can you rely on the compiler-generated copy constructor, and when must you write your own?

Complete the Code

Complete TODO 1 to TODO 3 below so that every copy is independent, and execute the program under -fsanitize=address.

```
#include <iostream>

#include <string>

class Prescription {

 int n;

 std::string* medicines; // dynamically allocated array

public:

 explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}

 // TODO 1: Copy constructor: allocate a NEW array of the same size and copy every  //
medicine (deep copy)


 // TODO 2: Copy assignment: guard against self-assignment, build a new array, copy,  //
release the old array and return *this


 ~Prescription()

 {

 // TODO 3: Release the dynamically allocated array


 }

 void set(int i, const std::string& m) { medicines[i] = m; }

 std::string get(int i) const { return medicines[i]; }

};

int main()
```

```
{
 Prescription p(2);
 p.set(0, "Paracetamol");
 p.set(1, "ORS");
 Prescription q = p; // copy constructor
 q.set(1, "Cetirizine");
 Prescription r(1);
 r = p; // copy assignment
 p.set(0, "Ibuprofen");
 std::cout << p.get(0) << ' ' << p.get(1) << " | " << q.get(0) << ' ' << q.get(1)
 << " | " << r.get(0) << ' ' << r.get(1) << '\n';
}
```

Expected output:

Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

Delete your copy constructor, run the program again under the sanitizer and record the error reported.

Alternative Concept and Complexity Analysis

If the medicine names were stored in a std::vector<std::string> instead of a dynamically allocated array, the compiler-generated copy constructor would already make a deep copy (the Rule of Zero). What would the solution be? Analyse the time O(n) and space O(n) to copy a prescription of n medicines in both designs, and compare them with a shallow copy (O(1)).

Q3 – Dynamic Constructors and Destructors 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Auditorium Seat Booking

An auditorium's seat map is created when an event is set up; the number of rows and seats per row differs from event to event. When the event object is destroyed, all memory must be released and a closing summary written.

Implementation Tasks

You must:

• Write a dynamic constructor that allocates a two-dimensional seat map with new • Write a destructor that releases every allocation and prints a summary • Create events on the stack, on the heap (new/delete) and inside a block • Book and cancel seats

Design & Analysis

You must:

• Explain the order in which constructors and destructors run for local, block-scope and dynamically created objects

• Explain what happens if delete is forgotten, and if delete is used instead of delete[] • Explain why the destructor takes no arguments and cannot be overloaded • Draw the memory layout of the seat map

Experimental Investigation

You must:

• Print messages in the constructor and destructor and record the order for each kind of object

• Run with -fsanitize=address, first with a missing delete and then with the correct code • Measure the memory allocated for 10 × 20, 100 × 200 and 1,000 × 2,000 seat maps • Test an event with zero rows

Conclusion

How does tying memory release to the destructor (the RAII idea) make the program safer?
Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

#include <string>

struct Logger {

 std::string n;

 Logger(const std::string& s) : n(s) { std::cout << '+' << n << ' '; }

 Logger(const Logger& o) : n(o.n + "c") { std::cout << "copy" << n << ' '; }  ~Logger() {
std::cout << '-' << n << ' '; }

};

void use(Logger) { std::cout << "use "; }

int main()

{
```

```
Logger a("A");
{
Logger b("B");
}
use(a);
Logger* p = new Logger("P");
delete p;
std::cout << "end ";
}
```

My predicted output (write before running):

Explain when each object was created and destroyed, and why the copy was made and destroyed around use().

Alternative Concept and Complexity Analysis

If the seat map were one block of rows × seats elements (index r × seats + s) or a std::vector<std::vector<char>>, instead of an array of row pointers each allocated with new, what would the solution be? Analyse the access time (O(1) in all), the number of allocations (1 versus rows + 1) and the extra space for the row pointers.

Q4 – Overloading Arithmetic and Stream Operators 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Money Arithmetic for a Billing Counter

A billing counter adds, subtracts and compares amounts of money stored in rupees and paise. The present code calls functions such as add(a, b) and print(c), which makes the bill calculation hard to read.

Implementation Tasks

You must:

• Write a Money class storing paise in a long long

• Overload +, -, +=, == and < as appropriate

• Overload << (and >>) for printing and reading amounts

• Write the bill calculation using the overloaded operators

Design & Analysis

You must:

• Explain why + returns a new object while += returns a reference to *this • Decide which operators are members and which are friends, and justify each • Explain why << must be a non-member function

• Show how a + b is translated by the compiler into a function call

Experimental Investigation

You must:

• Test addition that carries paise into rupees (Rs 0.75 + Rs 0.50) • Test subtraction that produces a negative amount and decide how to handle it • Compare the readability of the bill code with and without operator overloading • Compare the result with a double-based version for 1,000,000 additions

Engineering Decision

Should Money allow multiplication by another Money? Decide which operators make sense for money and which do not.

Complete the Code

Complete TODO 1 and TODO 2 below, and execute the program.

```
#include <iomanip>
#include <iostream>
class Money {
 long long paise;
public:
 Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}
 // TODO 1: Overload + as a member function that returns a new Money object
 // TODO 2: Overload == as a member, and << as a friend that prints "Rs 200.00"
};
int main()
{
 Money a(120, 75), b(79, 25);
 Money c = a + b;
 std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n'; }
```

Expected output:

Rs 200.00

true

Add a test where paise carry into rupees (Rs 0.75 + Rs 0.50).

Alternative Concept and Complexity Analysis

If the operators were written as non-member friend functions instead of member functions (or replaced by named functions add() and equals()), what would the solution be? Analyse whether time or space changes (all are O(1)), and explain which form allows 100 + m as well as m + 100.

Q5 – Overloading Increment, Comparison and Subscript Operators 6 Marks Bloom's Level: K5 – Evaluate

Problem Context – Music Playlist and Track Timer

A music player keeps a playlist of tracks and a Time object for the current position. The player needs time++ and ++time to advance by one second, comparisons between times, and playlist[i] to reach the i-th track with a range check.

Implementation Tasks

You must:

• Overload prefix and postfix ++ for Time

• Overload == and < for Time

• Overload [] for Playlist with a range check, in both a const and a non-const version • Use the operators in a short playback simulation

Design & Analysis

You must:

• Explain how the compiler tells prefix from postfix ++ (the int dummy parameter) • Explain why postfix ++ must return a copy and is slower

• Explain why operator[] needs a const and a non-const version

• Explain what should happen when an index is out of range

Experimental Investigation

You must:

• Verify the values of a and b after a = t++ and b = ++t

• Measure the time of 10,000,000 prefix and postfix increments at -O0 and -O2 • Test playlist[-1] and playlist[size]

• Test minute and hour roll-over (59:59 → 1:00:00)

Conclusion

Why do many programmers prefer ++i to i++ for class types?

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

struct Counter {

 int v;

 Counter& operator++() { ++v; return *this; } // prefix  Counter operator++(int) { Counter old = *this; ++v; return old; } // postfix };

int main()

{

 Counter c{5};

 Counter a = c++;

 Counter b = ++c;

 std::cout << a.v << ' ' << b.v << ' ' << c.v << '\n';

}
```

My predicted output (write before running):

Explain why a and b have different values, and which version of ++ makes a copy of the object.

Alternative Concept and Complexity Analysis

If playlist access used an at(i) function that throws std::out_of_range, or an unchecked operator[] like a built-in array, instead of operator[] with a range check that prints an error,

what would the solution be? Analyse the time per access (O(1) in all, with one extra comparison for checked access) and measure the cost of the check for 10,000,000 accesses.

Q6 – The Rules of Operator Overloading 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – A Simple Text Class for an SMS Gateway

An SMS gateway uses its own Text class (holding a dynamically allocated character array) so that it can control memory carefully. It needs assignment, concatenation with +, comparison and printing.

Implementation Tasks

You must:

• Implement Text with a constructor, copy constructor, copy assignment and destructor • Overload + for concatenation, == and != for comparison and << for printing • Prevent self-assignment problems

• Test long messages and empty messages

Design & Analysis

You must:

• List the operators that cannot be overloaded (::, ., .*, ?:, sizeof) and explain why

• Explain the rules: precedence and arity cannot change, at least one operand must be a user-defined type

• Explain why operator= must be a member function

• Explain when an overloaded operator should be a friend

Experimental Investigation

You must:

• Count the copies made when a + b + c is evaluated

• Run the program under the address sanitizer and fix any reported error • Compare your Text class with std::string for 100,000 concatenations • Test assigning a Text to itself

Conclusion

Your Text class works, but std::string exists. What did writing Text teach you, and when would you still write such a class?

Alternative Concept and Complexity Analysis

If the Text class were replaced by std::string, or if + were implemented with += on a copy (Text operator+(Text a, const Text& b) { a += b; return a; }), what would the solution be? Analyse the time to evaluate a + b + c for strings of lengths n1, n2, n3 (the characters copied) and the temporary space created in each version.

Q7 – Design a Complex-Number Class from First Principles 10 Marks Bloom's Level: K5 – Evaluate

Problem Context – AC Circuit Impedance Calculator

An electrical engineering laboratory needs a calculator for AC circuits. Impedances are complex numbers: resistors give R + 0j, inductors 0 + jωL and capacitors 0 − j/(ωC). Series impedances add; parallel impedances combine as (Z1 × Z2) / (Z1 + Z2). The

calculator must print the impedance in rectangular form and its magnitude, and must handle division by zero safely.

You are not given the design. You must develop it from the requirements. Stage 1 – Interpret the Problem | 2 Marks

• Identify the data and the operations needed

• Identify which constructors are needed (default, from real part, from real and imaginary parts)

• State the requirements as measurable statements

Stage 2 – Design | 3 Marks

• Design the Complex class with constructors and overloaded +, -, *, /, == and << • Decide which operators are members and which are friends

• Present the class design and the formula used for each operator

Stage 3 – Implement and Verify | 3 Marks

• Implement the class and compute the impedance of a series and a parallel RLC circuit

• Verify the results by hand calculation

• Test division by a zero impedance

Stage 4 – Defend the Design | 2 Marks

• Compare your class with std::complex<double>

• Explain why each constructor and operator is needed

• Identify one remaining limitation

Alternative Concept and Complexity Analysis

If std::complex<double> were used instead of your own Complex class, what would the solution be? Analyse the time per operation (O(1) in both) and the space (two doubles in both), and compare accuracy and the work needed to handle division by zero.

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.

• Your report containing the required evidence for Q1–Q7

• C++ source code for all questions

• Predictions, outputs, memory diagrams, sanitizer reports and timings

Assessment Verification

Students may be asked to explain, execute or modify any part of their design, code, tests or results during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

Q1 – Constructors and Constructor Overloading

Aim

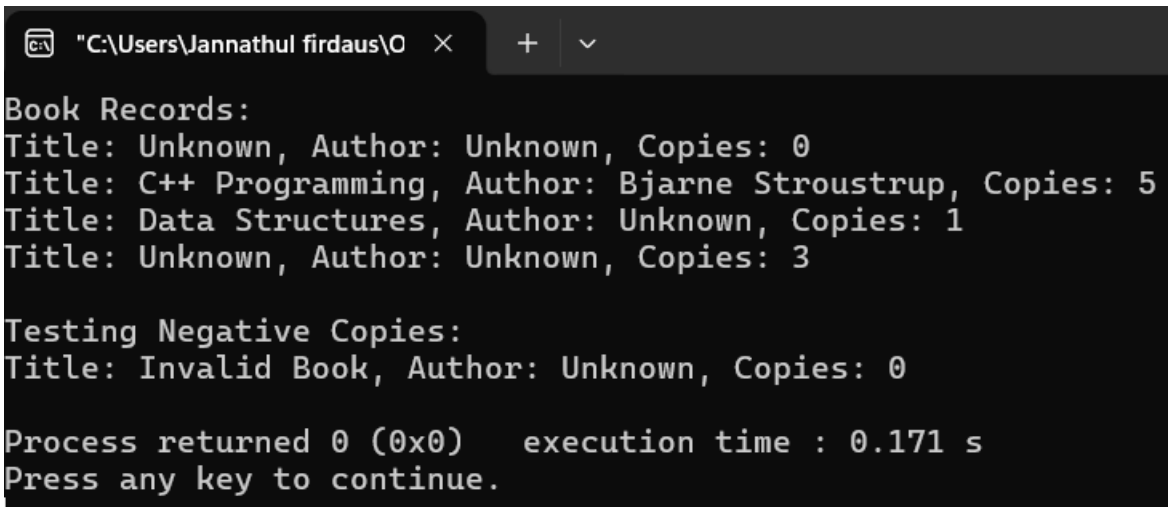
To implement a Book class using default, parameterised and overloaded constructors, member initializer lists and constructor delegation. To ensure that no Book object is created with an invalid number of copies.

C++ Program

```
#include <iostream>
#include <string>
using namespace std;
class Book {
    string title;
    string author;
    int copies;
public:
    // Default constructor
    Book() : Book("Unknown", "Unknown", 0) {}
    // Parameterized constructor
    Book(string t, string a, int c)
        : title(t), author(a), copies(c >= 0 ? c : 0) {}

    // Constructor for donated book
    Book(string t) : Book(t, "Unknown", 1) {}
    // Constructor for empty record with copies
    Book(int c) : Book("Unknown", "Unknown", c) {}
    // Display function
    void display() const {
```

```
        cout << "Title: " << title
             << ", Author: " << author
             << ", Copies: " << copies << endl;
    }
};
int main() {
    Book b1;
    Book b2("C++ Programming", "Bjarne Stroustrup", 5);
    Book b3("Data Structures");
    Book b4(3);
    cout << "Book Records:" << endl;
    b1.display();
    b2.display();
    b3.display();
    b4.display();
    // Testing negative copies
    Book b5("Invalid Book", "Unknown", -5);
    cout << "\nTesting Negative Copies:" << endl;
    b5.display();
    return 0;
}
```



Design and Analysis

a) When does the compiler supply a default constructor?

- The compiler automatically provides a default constructor if no constructor is defined by the programmer.
- If a parameterised constructor is defined, the compiler does not automatically provide a default constructor.
- Therefore, an explicit default constructor must be written when default object creation is required.

b) Why are member initializer lists preferred?

- They initialize data members directly during object construction.
- They avoid unnecessary assignment inside the constructor body.
- They are more efficient for class-type members.
- They are necessary for initializing const members and reference members.

c) Order of member initialization

- Data members are initialized in the order in which they are declared inside the class.
- The order in the member initializer list does not change the actual initialization order.
- In this program, the initialization order is title, author and copies.

d) State of three objects after construction

Object	Constructor used	Title	Author	Copies
b1	Default	Unknown	Unknown	0
b2	Parameterised	C++ Programming	Bjarne Stroustrup	5
b3	Overloaded	Data Structures	Unknown	1
b4	Integer Constructor	Unknown	Unknown	3

Experimental Investigation

a) Creating objects using every constructor

Objects were created using the default constructor, parameterised constructor and overloaded constructors. Their values were displayed successfully.

b) Removing the default constructor

The default constructor was removed, and an array of Book objects was created.

#include <iostream>

#include <string>

```
using namespace std;
class Book {
    string title;
    int copies;
public:
    Book(string t, int c) : title(t), copies(c) {}
};
int main() {
    Book books[3];
    return 0;
}
```

Compiler error obtained:

error: no matching function for call to 'Book::Book()'

note: candidate: 'Book::Book(std::string, int)'

Explanation: The compiler cannot create the array elements because no default constructor is available. The parameterised constructor requires two arguments.

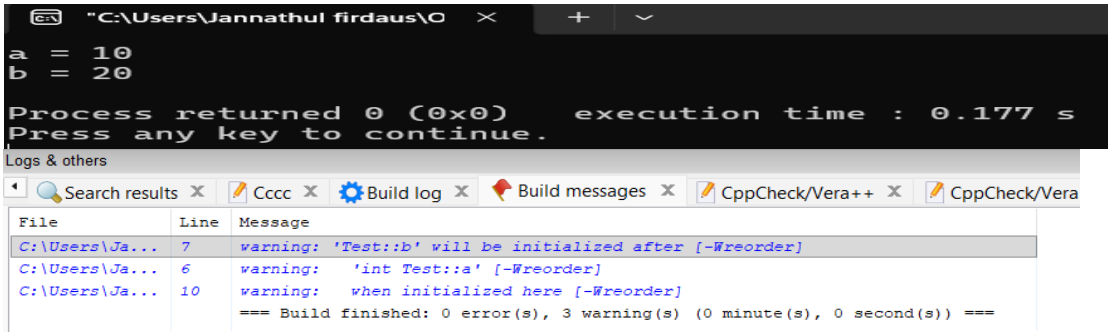
File	Line	Message
		=== Build file: "no target" in "no project" (compiler: unknown) ===
C:\Users\Ja...		In function 'int main()':
C:\Users\Ja...	20	error: no matching function for call to 'Book::Book()'
C:\Users\Ja...	12	note: candidate: 'Book::Book(std::string, int)'

c) Demonstrating wrong initialization order

```
#include <iostream>
using namespace std;
class Test {
    int a;
    int b;
public:
    Test() : b(20), a(10) {
        cout << "a = " << a << endl;
        cout << "b = " << b << endl;
```

```
    }
};
int main() {
    Test t;
    return 0;
}
```

Explanation: Although b is written first in the initializer list, a is initialized first because it is declared first in the class. The compiler generates a -Wreorder warning when warning flags are enabled.



d) Testing negative copies

Book b5("Invalid Book", "Unknown", -5);

Output:

Title: Invalid Book, Author: Unknown, Copies: 0

Explanation: The constructor checks whether the number of copies is negative. If it is negative, the value is replaced with zero. Therefore, an invalid number of copies is prevented.

Design Question – Explicit Constructor

The explicit keyword prevents implicit type conversion when a constructor accepts a single argument.

Without explicit:

```
class Book {
public:
    Book(string title) {}
};
Book b = "C++ Programming";
```

The statement compiles because implicit conversion is allowed.

With explicit:

```
class Book {
public:
    explicit Book(string title) {}
};
Book b = "C++ Programming";  // Compilation error
Book b("C++ Programming");   // Valid
```

An explicit constructor prevents unintended implicit conversions.

Alternative Concept and Complexity Analysis

Instead of using multiple overloaded constructors and constructor delegation, a single constructor with default arguments can be used.

Alternative implementation:

```
#include <iostream>
#include <string>
using namespace std;
class Book {
    string title;
    string author;
    int copies;
public:
    Book(string t = "Unknown",
         string a = "Unknown",
         int c = 0)
        : title(t), author(a), copies(c >= 0 ? c : 0) {}
    void display() const {
        cout << "Title: " << title
             << ", Author: " << author
             << ", Copies: " << copies << endl;
    }
```

};

Complexity Analysis

Criteria	**Constructor overloading**	**Default arguments**
Object creation time	O(1)	O(1)
Space complexity	O(1)	O(1)
Number of constructors	Multiple	One
Validation	Centralized through delegation	Single location
Code structure	Separate constructors	One flexible constructor

Evaluation:

Both approaches have O(1) time and O(1) space complexity for object creation. A single constructor with default arguments performs validation in one place and reduces code duplication. Constructor overloading with delegation provides separate interfaces for different book creation situations.

Q2. Copy Constructors: Shallow and Deep Copies

Aim

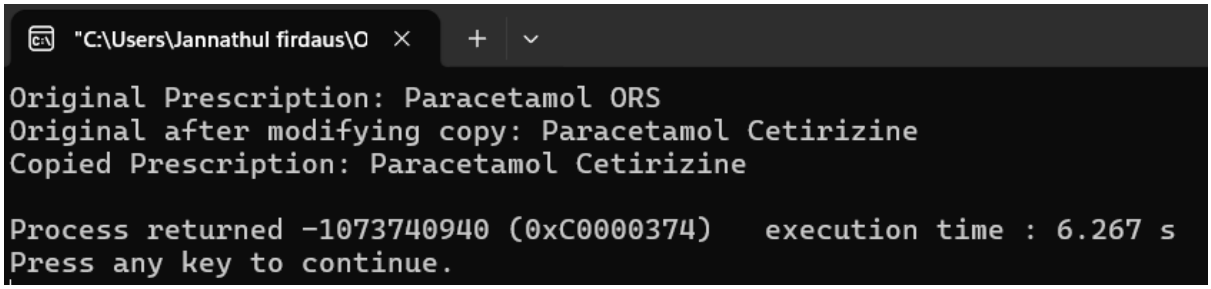
To understand shallow copying and deep copying in C++ using dynamically allocated memory, and to implement a copy constructor, copy assignment operator and destructor using the Rule of Three.

Implementation of Shallow Copy (Without Copy Constructor)

Code – Shallow Copy

```
#include <iostream>
#include <string>
using namespace std;
class Prescription {
   int n;
   string* medicines;
public:
   Prescription(int count) : n(count) {
      medicines = new string[n];
   }
   ~Prescription() {
```

```
        delete[] medicines;
    }
    void set(int i, const string& m) {
        medicines[i] = m;
    }
    string get(int i) const {
        return medicines[i];
    }
};
int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    cout << "Original Prescription: "<< p.get(0) << " " << p.get(1) << endl;
    Prescription q = p;
    q.set(1, "Cetirizine");
    cout << "Original after modifying copy: " << p.get(0) << " " << p.get(1) << endl;
    cout << "Copied Prescription: " << q.get(0) << " " << q.get(1) << endl;
    return 0;
}
```

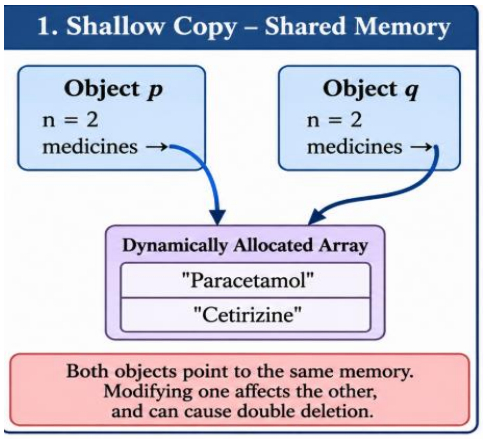


Explanation

The copied object q and original object p share the same dynamically allocated array. Therefore, modifying the copied object also changes the original object.

When both objects are destroyed, their destructors attempt to delete the same memory. This causes a double deletion and heap corruption.

Memory Diagram – Shallow Copy



Implementation of Deep Copy Using Rule of Three

To avoid shared memory and double deletion, a deep copy is implemented.

The Rule of Three states that if a class requires a user-defined destructor, copy constructor or copy assignment operator, it generally requires all three.

TODO 1 – Deep Copy Constructor

The copy constructor allocates a new array and copies each medicine name from the source object. This ensures that the copied object has independent memory.

TODO 2 – Copy Assignment Operator

The copy assignment operator checks for self-assignment, allocates new memory, copies the medicine names, releases the old memory and returns the current object.

TODO 3 – Destructor

The destructor releases the dynamically allocated array using delete[] to prevent memory leaks.

```
#include <iostream>

#include <string>

class Prescription {

    int n;

    std::string* medicines;

public:

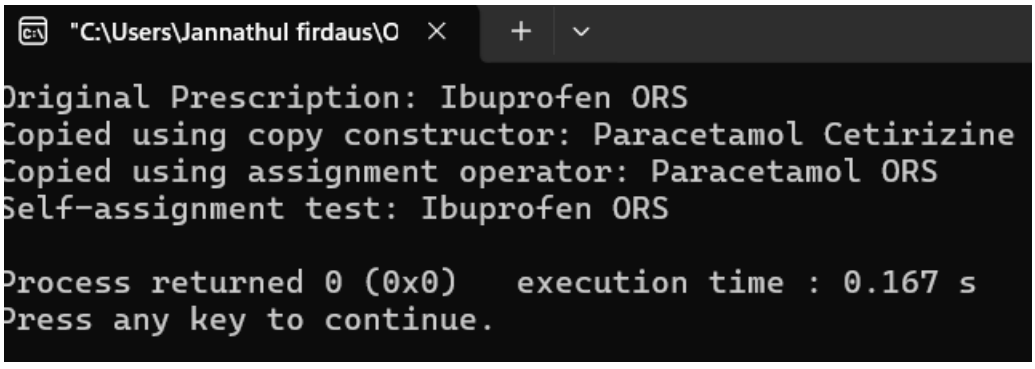
    explicit Prescription(int count)

        : n(count), medicines(new std::string[count]) {}

    // TODO 1: Deep copy constructor
```

```
Prescription(const Prescription& other)
    : n(other.n), medicines(new std::string[other.n]) {
    for (int i = 0; i < n; i++) {
        medicines[i] = other.medicines[i];
    }
}
// TODO 2: Copy assignment operator
Prescription& operator=(const Prescription& other) {
    if (this != &other) {
        std::string* temp = new std::string[other.n];
        for (int i = 0; i < other.n; i++) {
            temp[i] = other.medicines[i];
        }
        delete[] medicines;
        medicines = temp;
        n = other.n;
    }
    return *this;
}
// TODO 3: Destructor
~Prescription() {
    delete[] medicines;
}
void set(int i, const std::string& m) {
    medicines[i] = m;
}
std::string get(int i) const {
    return medicines[i];
}
```

```
};
int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    // Copy constructor
    Prescription q = p;
    q.set(1, "Cetirizine");
    // Copy assignment operator
    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");
    // Self-assignment test
    p = p;
    std::cout << "Original Prescription: " << p.get(0) << " " << p.get(1) << '\n';
    std::cout << "Copied using copy constructor: "  << q.get(0) << " " << q.get(1) << '\n';
    std::cout << "Copied using assignment operator: " << r.get(0) << " " << r.get(1) << '\n';
    std::cout << "Self-assignment test: "<< p.get(0) << " " << p.get(1) << '\n';
    return 0;
}
```



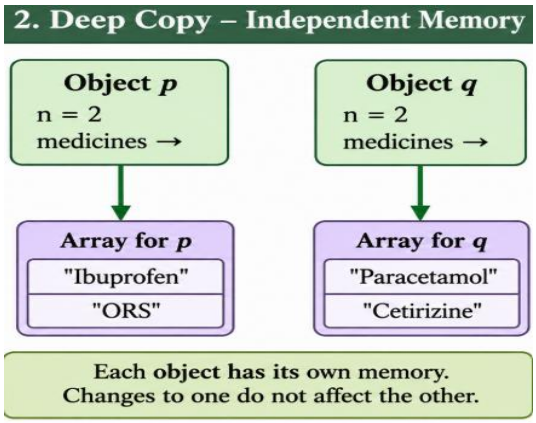
Explanation

The deep copy constructor creates a separate array for the copied object. Therefore, changes to one prescription do not affect another.

The copy assignment operator safely handles assignment and prevents self-assignment from deleting the object's own data.

The destructor releases the memory allocated by each object independently. Thus, double deletion is avoided.

Memory Diagram – Deep Copy



Copy Constructor Experiment

A copy constructor is called in the following situations:

1. When an object is initialized using another object, for example Prescription q = p;.
2. When an object is passed to a function by value.
3. When an object is returned from a function by value, subject to compiler copy elision.

Passing by Value and Passing by Const Reference

The copy constructor is called when an object is passed by value because a separate parameter object is created. Passing by const reference does not create a copy.

Experimental Results

Experiment	Number of Copies
Passed by value	1
Passed by const reference	0

Copying Time Experiment

The time required to copy prescriptions of different sizes was measured.

Experimental Results

Number of Medicines	Repetitions	Total Time
10	10,000	8,557 microseconds

1,000	1,000	12,352 microseconds
100,000	10	16,149 microseconds

The measured results show the time taken for deep copying at different prescription sizes. The time generally depends on the number of medicines, although the measurements are also affected by repetitions and system execution conditions.

Time and Space Complexity

- Shallow copy: O(1) time and O(1) additional space because only pointer values are copied.
- Deep copy: O(n) time and O(n) additional space because a new array is allocated and all elements are copied.

Self-Assignment Test

The statement p = p; was executed to test self-assignment.

Result: The original prescription remained Ibuprofen ORS.

The self-assignment check if (this != &other) prevents the object from deleting its own data before copying it.

1. Shallow copy – AddressSanitizer test

```
#include <iostream>
#include <string>
using namespace std;
class Prescription {
   int n;
   string* medicines;
public:
   Prescription(int count) : n(count) {
      medicines = new string[n];
   }
   // No copy constructor: compiler-generated shallow copy
   ~Prescription() {
      delete[] medicines;
   }
   void set(int i, const string& m) {
```

```
        medicines[i] = m;
    }
    string get(int i) const {
        return medicines[i];
    }
};
int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;  // Shallow copy
    q.set(1, "Cetirizine");
    cout << "Original: " << p.get(0)
         << " " << p.get(1) << endl;
    cout << "Copied: " << q.get(0)
         << " " << q.get(1) << endl;
    return 0;
}
```

Expected output:

Original: Paracetamol Cetirizine

Copied: Paracetamol Cetirizine

ERROR: AddressSanitizer: attempting double-free

on an address previously allocated by thread T0

SUMMARY: AddressSanitizer: double-free

2. Deep copy – AddressSanitizer test

```
#include <iostream>
#include <string>
using namespace std;
class Prescription {
```

```
    int n;
    string* medicines;
public:
    Prescription(int count) : n(count) {
        medicines = new string[n];
    }
    // Deep copy constructor
    Prescription(const Prescription& other) {
        n = other.n;
        medicines = new string[n];
        for (int i = 0; i < n; i++) {
            medicines[i] = other.medicines[i];
        }
    }
    // Copy assignment operator
    Prescription& operator=(const Prescription& other) {
        if (this != &other) {
            string* newMedicines = new string[other.n];
            for (int i = 0; i < other.n; i++) {
                newMedicines[i] = other.medicines[i];
            }
            delete[] medicines;
            medicines = newMedicines;
            n = other.n;
        }
        return *this;
    }
    // Destructor
    ~Prescription() {
```

```
        delete[] medicines;
    }
    void set(int i, const string& m) {
        medicines[i] = m;
    }
    string get(int i) const {
        return medicines[i];
    }
};
int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;
    q.set(1, "Cetirizine");
    cout << "Original: " << p.get(0)
         << " " << p.get(1) << endl;
    cout << "Copied: " << q.get(0)
         << " " << q.get(1) << endl;
    return 0;
}
```

Expected output:

Original: Paracetamol ORS

Copied: Paracetamol Cetirizine

Alternative Using std::vector

Instead of managing a dynamic array manually, std::vector<std::string> can be used.

The vector manages its own memory, and its compiler-generated copy constructor performs an independent copy of the elements. This follows the Rule of Zero, where the class does not need to define its own destructor, copy constructor or copy assignment operator for memory management.

Q3: Dynamic Constructors and Destructors

Aim

To implement dynamic constructors and destructors in C++ using dynamic memory allocation. The program manages an auditorium seat map, books and cancels seats, and releases allocated memory automatically when objects are destroyed.

C++ Program

```
#include <iostream>
#include <string>
using namespace std;
class Event {
    string eventName;
    int rows, seats;
    char** seatMap;
    int booked;
public:
    Event(string name, int r, int s)
        : eventName(name), rows(r), seats(s),
          seatMap(nullptr), booked(0) {
        if (rows > 0 && seats > 0) {
            seatMap = new char*[rows];
            for (int i = 0; i < rows; i++) {
                seatMap[i] = new char[seats];
                for (int j = 0; j < seats; j++) {
                    seatMap[i][j] = 'O';
                }
            }
        }
        cout << "Constructor: " << eventName << endl;
```

```
}
void bookSeat(int r, int s) {
    if (r < 0 || r >= rows || s < 0 || s >= seats) {
        cout << "Invalid seat position" << endl;
    }
    else if (seatMap[r][s] == 'X') {
        cout << "Seat already booked" << endl;
    }
    else {
        seatMap[r][s] = 'X';
        booked++;
        cout << "Seat booked: " << r << ", " << s << endl;
    }
}
void cancelSeat(int r, int s) {
    if (r < 0 || r >= rows || s < 0 || s >= seats) {
        cout << "Invalid seat position" << endl;
    }
    else if (seatMap[r][s] == 'O') {
        cout << "Seat is not booked" << endl;
    }
    else {
        seatMap[r][s] = 'O';
        booked--;
        cout << "Booking cancelled: " << r << ", " << s << endl;
    }
}
void display() const {
    cout << "Seat Map - " << eventName << endl;
```

```
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < seats; j++)
                cout << seatMap[i][j] << " ";
            cout << endl;
        }
    }

    size_t memoryAllocated() const {
        return rows * sizeof(char*) + rows * seats * sizeof(char);
    }
    ~Event() {
        if (seatMap != nullptr) {
            for (int i = 0; i < rows; i++)
                delete[] seatMap[i];
            delete[] seatMap;
        }
        cout << "Destructor: " << eventName
             << " | Booked seats: " << booked
             << " | Memory released" << endl;
    }
};
int main() {
    cout << "--- Stack Object ---" << endl;
    Event stackEvent("Stack Event", 2, 3);
    stackEvent.bookSeat(0, 1);
    stackEvent.bookSeat(1, 2);
    stackEvent.cancelSeat(0, 1);
    stackEvent.display();
    cout << "\n--- Block Scope Object ---" << endl;
```

```
    {
        Event blockEvent("Block Event", 2, 2);
        blockEvent.bookSeat(0, 0);
    }
    cout << "Exited block" << endl;
    cout << "\n--- Heap Object ---" << endl;
    Event* heapEvent = new Event("Heap Event", 3, 3);
    heapEvent->bookSeat(1, 1);
    heapEvent->display();
    delete heapEvent;
    cout << "Heap object deleted" << endl;
    cout << "\n--- Memory Measurement ---" << endl;
    Event e1("10x20 Event", 10, 20);
    Event e2("100x200 Event", 100, 200);
    Event e3("1000x2000 Event", 1000, 2000);
    cout << "10 x 20: " << e1.memoryAllocated() << " bytes" << endl;
    cout << "100 x 200: " << e2.memoryAllocated() << " bytes" << endl;
    cout << "1000 x 2000: " << e3.memoryAllocated() << " bytes" << endl;
    cout << "\n--- Zero Rows Test ---" << endl;
    Event zeroEvent("Zero Row Event", 0, 5);
    cout << "Memory allocated: "
         << zeroEvent.memoryAllocated() << " bytes" << endl;
    cout << "\nEnd of main" << endl;
    return 0;
}
```

```
"C:\Users\Jannathul firdaus\C
--- Stack Object ---
Constructor: Stack Event
Seat booked: 0, 1
Seat booked: 1, 2
Booking cancelled: 0, 1
Seat Map - Stack Event
O O O
O O X

--- Block Scope Object ---
Constructor: Block Event
Seat booked: 0, 0
Destructor: Block Event | Booked seats: 1 | Memory released
Exited block

--- Heap Object ---
Constructor: Heap Event
Seat booked: 1, 1
Seat Map - Heap Event
O O O
O X O
O O O
Destructor: Heap Event | Booked seats: 1 | Memory released
Heap object deleted

--- Memory Measurement ---
Constructor: 10x20 Event
Constructor: 100x200 Event
Constructor: 1000x2000 Event
10 x 20: 280 bytes
```

```
--- Zero Rows Test ---
Constructor: Zero Row Event
Memory allocated: 0 bytes

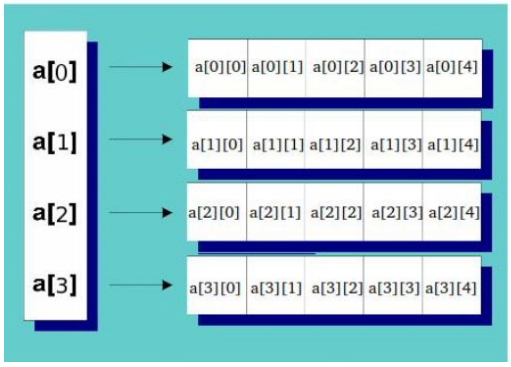
End of main
Destructor: Zero Row Event | Booked seats: 0 | Memory released
Destructor: 1000x2000 Event | Booked seats: 0 | Memory released
Destructor: 100x200 Event | Booked seats: 0 | Memory released
Destructor: 10x20 Event | Booked seats: 0 | Memory released
Destructor: Stack Event | Booked seats: 1 | Memory released

Process returned 0 (0x0)   execution time : 0.179 s
Press any key to continue.
```

Memory Layout Diagram

```
int** a = new int*[rowCount];
for(int i = 0; i < rowCount; ++i)
    a[i] = new int[colCount];
```

The above, for `colCount= 5` and `rowCount = 4`, would produce the following:



Constructor and Destructor Execution Order

The program demonstrates three object-lifetime situations:

Object	Construction	Destruction
Stack Event	When declared in main()	At the end of main()
Block Event	When entering the block	When leaving the block
Heap Event	Using new	When delete is executed

Important observations:

1. Constructors execute when objects are created.
2. Block-scope objects are destroyed automatically when their block ends.
3. Heap objects remain allocated until explicitly deleted.

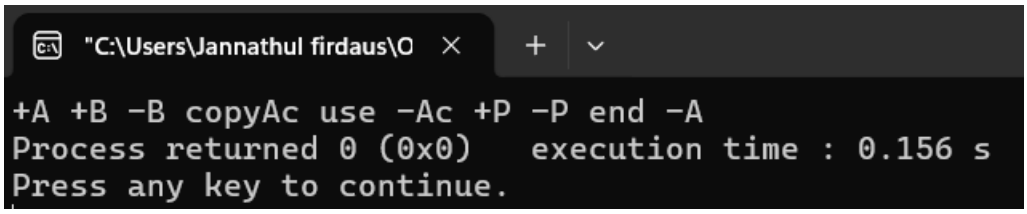
4. Local objects are destroyed in reverse order of their declaration.
5. The destructor releases dynamically allocated memory.

Constructor and Destructor Order Prediction

```
#include <iostream>
#include <string>
using namespace std;
struct Logger {
   string n;
   Logger(const string& s) : n(s) {
      cout << '+' << n << ' ';
   }
   Logger(const Logger& o) : n(o.n + "c") {
      cout << "copy" << n << ' ';
   }
   ~Logger() {
      cout << '-' << n << ' ';
   }
};
void use(Logger) {
   cout << "use ";
}
int main() {
   Logger a("A");

   {
      Logger b("B");
   }
   use(a);
   Logger* p = new Logger("P");
```

```
    delete p;
    cout << "end ";
}
```



Explanation:

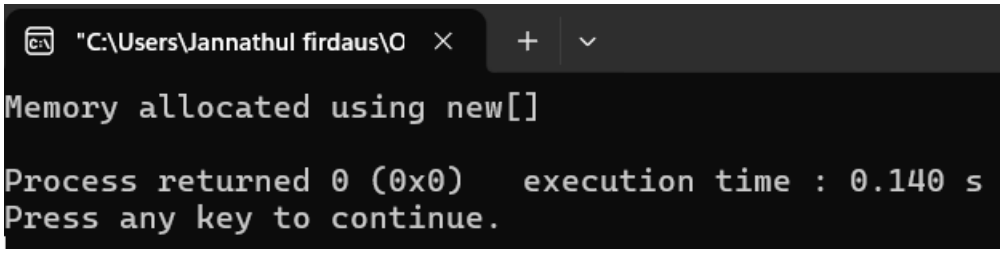
- A is constructed first.
- B is constructed and destroyed when its block ends.
- use(a) creates a copy named Ac; it is destroyed after the function call.
- P is dynamically created and destroyed by delete.
- A is destroyed last when main() ends.

Memory Leak Experiment

A. Program without delete[]

```
#include <iostream>
using namespace std;
int main() {
    int* data = new int[100];
    for (int i = 0; i < 100; i++) {
        data[i] = i;
    }
    cout << "Memory allocated using new[]" << endl;
    // Intentionally missing delete[].
    return 0;
}
```

Actual output:



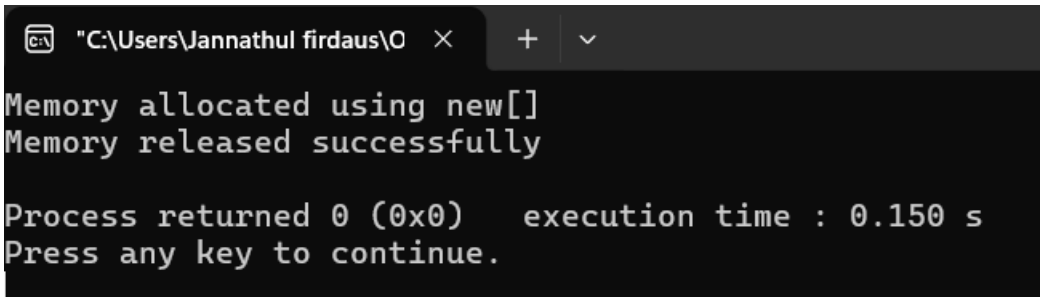
The program allocates memory for 100 integers but does not explicitly release it. This causes a memory leak.

B. Program with delete[]

```
#include <iostream>

using namespace std;

int main() {
    int* data = new int[100];
    for (int i = 0; i < 100; i++) {
        data[i] = i;
    }
    cout << "Memory allocated using new[]" << endl;
    delete[] data;
    cout << "Memory released successfully" << endl;
    return 0;
}
```



The delete[] operator releases the allocated array memory and prevents the memory leak.

Expected AddressSanitizer results:

Test	**Expected result**
Missing delete[]	LeakSanitizer reports a 400-byte memory leak
Correct delete[]	No memory leak report

The expected leak is calculated as:

100 x sizeof(int) = 100 x 4 = 400 text bytes

Memory Measurement Analysis

The program measures the memory required for different auditorium sizes.

Auditorium size	Row pointers	Seat storage	Total
10 × 20	80 bytes	200 bytes	280 bytes
100 × 200	800 bytes	20,000 bytes	20,800 bytes
1000 × 2000	8,000 bytes	2,000,000 bytes	2,008,000 bytes

The calculation assumes a 64-bit system, where a pointer occupies 8 bytes and a character occupies 1 byte. These figures exclude allocator overhead.

Q4: Overloading Arithmetic and Stream Operators

Aim

To implement operator overloading in C++ using a Money class that stores monetary values in paise. The program performs arithmetic operations, comparisons, input/output operations and billing calculations using overloaded operators.

C++ Program

```
#include <iostream>
#include <iomanip>
#include <chrono>
#include <cmath>
#include <cstdlib>
using namespace std;
class Money {
   long long paise;
public:
   Money(long long rupees = 0, int p = 0)
      : paise(rupees * 100 + p) {}
   Money operator+(const Money& other) const {
      return Money(0, paise + other.paise);
   }
```

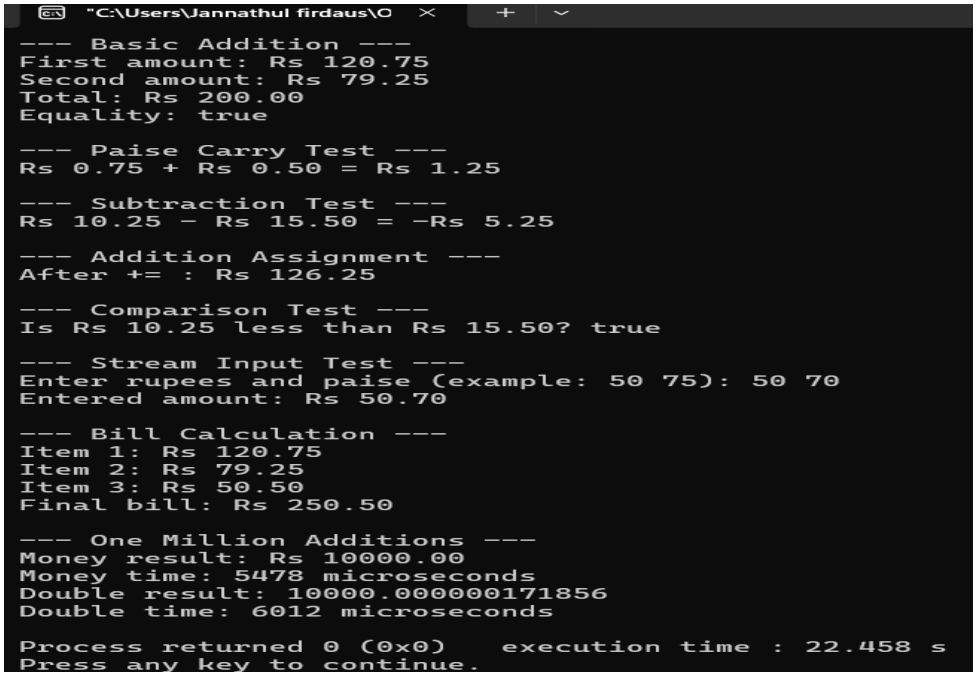
```
Money operator-(const Money& other) const {
    return Money(0, paise - other.paise);
}

Money& operator+=(const Money& other) {
    paise += other.paise;
    return *this;
}
bool operator==(const Money& other) const {
    return paise == other.paise;
}
bool operator<(const Money& other) const {
    return paise < other.paise;
}
friend ostream& operator<<(ostream& out, const Money& m) {
    long long amount = m.paise;
    if (amount < 0) {
        out << "-Rs ";
        amount = -amount;
    }
    else {
        out << "Rs ";
    }
    out << amount / 100 << "."
        << setw(2) << setfill('0') << amount % 100
        << setfill(' ');
    return out;
}
friend istream& operator>>(istream& in, Money& m) {
```

```
        long long rupees;
        int p;
        if (in >> rupees >> p) {
            m.paise = rupees * 100 + p;
        }
        return in;
    }
};
int main() {
    cout << "--- Basic Addition ---" << endl;
    Money a(120, 75);
    Money b(79, 25);
    Money c = a + b;
    cout << "First amount: " << a << endl;
    cout << "Second amount: " << b << endl;
    cout << "Total: " << c << endl;
    cout << "Equality: " << boolalpha
         << (c == Money(200)) << endl;
    cout << "\n--- Paise Carry Test ---" << endl;
    Money x(0, 75);
    Money y(0, 50);
    Money carry = x + y;
    cout << x << " + " << y << " = " << carry << endl;
    cout << "\n--- Subtraction Test ---" << endl;
    Money d(10, 25);
    Money e(15, 50);
    Money difference = d - e;
    cout << d << " - " << e
         << " = " << difference << endl;
```

```
cout << "\n--- Addition Assignment ---" << endl;
Money total(100, 50);
total += Money(25, 75);
cout << "After += : " << total << endl;
cout << "\n--- Comparison Test ---" << endl;
cout << "Is Rs 10.25 less than Rs 15.50? "
    << (d < e) << endl;
cout << "\n--- Stream Input Test ---" << endl;
Money input;
cout << "Enter rupees and paise (example: 50 75): ";
if (cin >> input) {
   cout << "Entered amount: " << input << endl;
}
cout << "\n--- Bill Calculation ---" << endl;
Money item1(120, 75);
Money item2(79, 25);
Money item3(50, 50);
Money bill;
bill += item1;
bill += item2;
bill += item3;
cout << "Item 1: " << item1 << endl;
cout << "Item 2: " << item2 << endl;
cout << "Item 3: " << item3 << endl;
cout << "Final bill: " << bill << endl;
cout << "\n--- One Million Additions ---" << endl;
const int N = 1000000;
Money moneyTotal;
```

```
    auto start1 = chrono::high_resolution_clock::now();
    for (int i = 0; i < N; i++) {
        moneyTotal += Money(0, 1);
    }
    auto end1 = chrono::high_resolution_clock::now();
    long long moneyTime =
        chrono::duration_cast<chrono::microseconds>(
            end1 - start1).count();
    cout << "Money result: " << moneyTotal << endl;
    cout << "Money time: " << moneyTime
         << " microseconds" << endl;
    double doubleTotal = 0.0;
    auto start2 = chrono::high_resolution_clock::now();
    for (int i = 0; i < N; i++) {
        doubleTotal += 0.01;
    }
    auto end2 = chrono::high_resolution_clock::now();
    long long doubleTime =
        chrono::duration_cast<chrono::microseconds>(
            end2 - start2).count();
    cout << fixed << setprecision(12);
    cout << "Double result: " << doubleTotal << endl;
    cout << "Double time: " << doubleTime
        << " microseconds" << endl;
    return 0;
}
```



Operator Overloading Explanation

Operator	Purpose	Implementation
+	Adds two Money objects	Member function
-	Subtracts two Money objects	Member function
+=	Adds and updates current object	Member function
==	Checks equality	Member function
<	Compares two amounts	Member function
<<	Displays an amount	Friend function
>>	Reads an amount	Friend function

Why does + return a new object?

The + operator performs addition without modifying the original operands. Therefore, it returns a new Money object containing the result.

Example:

Money c = a + b;

The original objects a and b remain unchanged.

Why does += return a reference to *this?

The += operator modifies the existing object. Returning Money& refers to the same object and avoids creating an unnecessary copy.

Money& operator+=(const Money& other) {

```
    paise += other.paise;
    return *this;
}
```

Why is << a non-member friend function?

The left operand of the stream insertion operator is std::ostream, not a Money object. Therefore, it cannot be implemented as a normal member function of Money with the expected expression:

cout << c;

A friend function is used to access the private paise member while keeping Money as the right operand.

How the Compiler Translates Operators

The expression:

Money c = a + b;

For the overloaded member operator, the compiler interprets it as:

Money c = a.operator+(b);

Similarly:

a += b;

is interpreted as:

a.operator+=(b);

For stream insertion:

cout << c;

the corresponding function call is:

operator<<(cout, c);

Experimental Investigation

A. Paise carry test

The program adds:

$$Rs\ 0.75 + Rs\ 0.50 = Rs\ 1.25$$

The result is correct because the class stores the complete amount in paise:

$$75 + 50 = 125\ paise$$

$$125\ paise = Rs\ 1.25$$

B. Negative subtraction test

The program calculates:

$$Rs\ 10.25 - Rs\ 15.50 = -Rs\ 5.25$$

The program allows negative amounts and displays the negative sign. This is useful for representing refunds, credits and outstanding balances.

C. Readability comparison

Without operator overloading:

Money c = add(a, b);

print(c);

With operator overloading:

Money c = a + b;

cout << c;

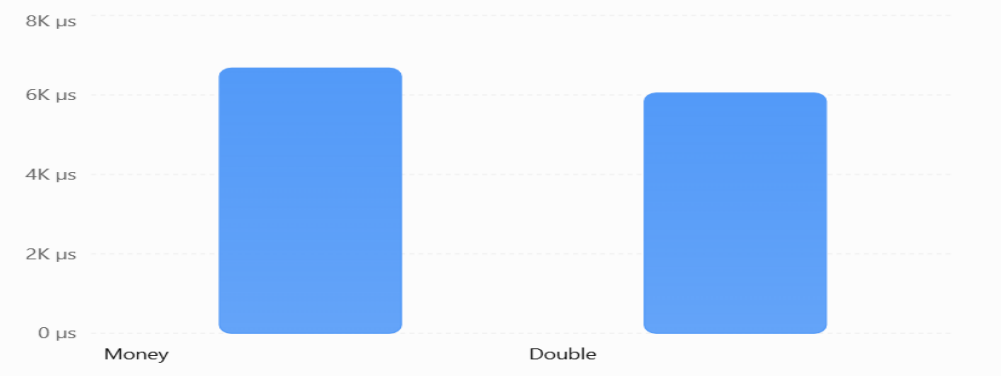
Operator overloading makes the code shorter, more readable and similar to ordinary mathematical expressions.

Performance Comparison: Money vs Double

The program performs one million additions using both integer-based Money and double.

Measurement	**Money class**	**Double**
Number of additions	1,000,000	1,000,000
Final result	Rs 10000.00	10000.000000171856
Execution time	6675 µs	6048 µs
Precision	Exact to paise for supported range	Floating-point approximation

Execution time comparison



Analysis:

- The Money class stores values as integer paise, avoiding binary floating-point representation errors.
- The double-based result contains a small precision difference.
- Double took 6048 microseconds, while Money took 6675 microseconds in this run.
- The measured difference is small and can vary with compiler optimization, hardware and system load.
- Both approaches have (O(1)) time per addition and (O(1)) additional space.

Integer paise is more suitable for ordinary billing calculations because exact currency representation is important.

Alternative Concept and Complexity Analysis

Operators can also be implemented as non-member friend functions or replaced with named functions.

A. Non-member friend implementation

```
friend Money operator+(const Money& a, const Money& b) {
   return Money(0, a.paise + b.paise);
}
friend bool operator==(const Money& a, const Money& b) {
   return a.paise == b.paise;
}
```

Both operands are passed explicitly. This approach allows more flexible operand conversions.

B. Named function implementation

```
Money add(const Money& a, const Money& b) {
   return Money(0, a.getPaise() + b.getPaise());
}
bool equals(const Money& a, const Money& b) {
   return a.getPaise() == b.getPaise();
}
```

Here, suitable public getter functions must be provided to access the private amount.

C. Complexity comparison

Implementation	Time complexity	Additional space

Member operator	O(1)	O(1)
Friend operator	O(1)	O(1)
Named function	O(1)	O(1)

The implementation style does not change the asymptotic time or space complexity.

D. Supporting 100 + m **and** m + 100

A member operator is called on the left operand. Therefore, m + 100 can work if an appropriate implicit conversion from int to Money is available.

However, 100 + m cannot use Money's member operator+, because the left operand is an integer.

A non-member operator can support both operand orders by defining suitable overloads and conversions:

```
friend Money operator+(const Money& a, const Money& b);

friend Money operator+(long long rupees, const Money& m) {
   return Money(rupees) + m;
}

friend Money operator+(const Money& m, long long rupees) {
   return Money(rupees) + m;
}
```

This demonstrates the flexibility of non-member operator overloading.

Engineering Decision

Multiplication of one Money object by another Money object should generally **not** be supported.

For example:

$$Rs\ 100 \times Rs\ 50$$

produces a squared currency unit, which has no normal meaning in billing.

Appropriate operators include:

- + for adding amounts.
- - for subtracting amounts.
- += for updating a bill.
- == for checking equal amounts.

- < for comparing amounts.
- * with a numeric quantity for calculating the total price of multiple items.
- / with a numeric quantity for splitting an amount, with a clearly defined rounding policy.

Multiplication between two Money objects is not meaningful for ordinary financial calculations.

Q5: Overloading Increment, Comparison and Subscript Operators

Aim

To implement operator overloading in C++ using a Time class and a Playlist class. The program demonstrates prefix and postfix increment, comparison operators, checked array subscript access, and a music playback simulation.

C++ Program

```
#include <iostream>
#include <iomanip>
#include <vector>
#include <string>
#include <stdexcept>
#include <chrono>
using namespace std;
class Time {
   long long seconds;
public:
   Time(int h = 0, int m = 0, int s = 0) {
      seconds = h * 3600LL + m * 60LL + s;
   }
   // Prefix increment
   Time& operator++() {
      ++seconds;
      return *this;
   }
```

```
    // Postfix increment
    Time operator++(int) {
        Time old = *this;
        ++seconds;
        return old;
    }
    bool operator==(const Time& other) const {
        return seconds == other.seconds;
    }
    bool operator<(const Time& other) const {
        return seconds < other.seconds;
    }
    long long totalSeconds() const {
        return seconds;
    }
    void display() const {
        long long h = seconds / 3600;
        long long m = (seconds % 3600) / 60;
        long long s = seconds % 60;
        cout << setfill('0')
             << setw(2) << h << ":"
             << setw(2) << m << ":"
             << setw(2) << s
             << setfill(' ');
    }
};
class Playlist {
    vector<string> tracks;
public:
```

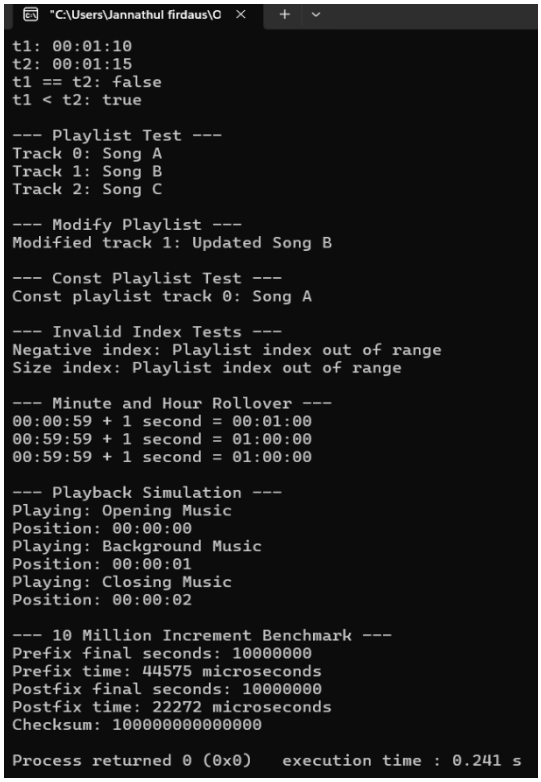
```
    void addTrack(const string& name) {
        tracks.push_back(name);
    }
    // Non-const subscript operator
    string& operator[](int index) {
        if (index < 0 || index >= (int)tracks.size()) {
            throw out_of_range("Playlist index out of range");
        }
        return tracks[index];
    }
    // Const subscript operator
    const string& operator[](int index) const {
        if (index < 0 || index >= (int)tracks.size()) {
            throw out_of_range("Playlist index out of range");
        }
        return tracks[index];
    }
    int size() const {
        return (int)tracks.size();
    }
};
int main() {
    cout << "--- Prefix and Postfix Test ---" << endl;
    Time t(0, 0, 5);
    Time a = t++;
    Time b = ++t;
    cout << "Initial time: 00:00:05" << endl;
    cout << "a after t++: ";
    a.display();
```

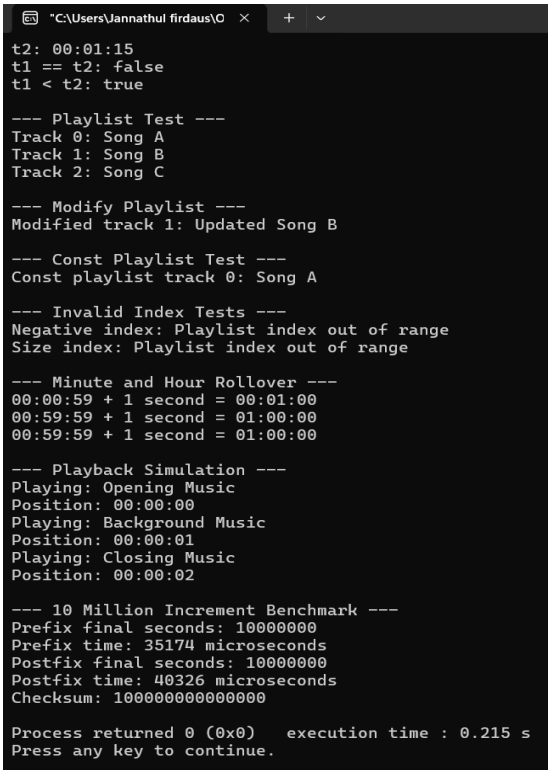
```
cout << endl;
cout << "b after ++t: ";
b.display();
cout << endl;
cout << "Current t: ";
t.display();
cout << endl;
cout << "\n--- Time Comparison ---" << endl;
Time t1(0, 1, 10);
Time t2(0, 1, 15);
cout << "t1: ";
t1.display();
cout << "\nt2: ";
t2.display();
cout << "\nt1 == t2: " << boolalpha << (t1 == t2);
cout << "\nt1 < t2: " << (t1 < t2) << endl;
cout << "\n--- Playlist Test ---" << endl;
Playlist playlist;
playlist.addTrack("Song A");
playlist.addTrack("Song B");
playlist.addTrack("Song C");
for (int i = 0; i < playlist.size(); i++) {
    cout << "Track " << i << ": "
         << playlist[i] << endl;
}
cout << "\n--- Modify Playlist ---" << endl;
playlist[1] = "Updated Song B";
cout << "Modified track 1: "
     << playlist[1] << endl;
```

```
cout << "\n--- Const Playlist Test ---" << endl;
const Playlist& constPlaylist = playlist;
cout << "Const playlist track 0: "
     << constPlaylist[0] << endl;
cout << "\n--- Invalid Index Tests ---" << endl;
try {
    cout << playlist[-1] << endl;
}
catch (const out_of_range& e) {
    cout << "Negative index: " << e.what() << endl;
}
try {
    cout << playlist[playlist.size()] << endl;
}
catch (const out_of_range& e) {
    cout << "Size index: " << e.what() << endl;
}
cout << "\n--- Minute and Hour Rollover ---" << endl;
Time minuteTest(0, 0, 59);
++minuteTest;
cout << "00:00:59 + 1 second = ";
minuteTest.display();
cout << endl;
Time hourTest(0, 59, 59);
++hourTest;
cout << "00:59:59 + 1 second = ";
hourTest.display();
cout << endl;
Time dayTest(0, 59, 59);
```

```
++dayTest;
cout << "00:59:59 + 1 second = ";
dayTest.display();
cout << endl;
cout << "\n--- Playback Simulation ---" << endl;
Playlist music;
music.addTrack("Opening Music");
music.addTrack("Background Music");
music.addTrack("Closing Music");
Time position(0, 0, 0);
for (int i = 0; i < music.size(); i++) {
    cout << "Playing: " << music[i] << endl;
    cout << "Position: ";
    position.display();
    cout << endl;
    ++position;
}
cout << "\n--- 10 Million Increment Benchmark ---" << endl;
const int N = 10000000;
volatile long long checksum = 0;
Time prefixTime;
auto startPrefix = chrono::high_resolution_clock::now();
for (int i = 0; i < N; i++) {
    ++prefixTime;
    checksum = checksum + prefixTime.totalSeconds();
}
auto endPrefix = chrono::high_resolution_clock::now();
long long prefixDuration =
    chrono::duration_cast<chrono::microseconds>(
```

```
        endPrefix - startPrefix).count();
    cout << "Prefix final seconds: "
        << prefixTime.totalSeconds() << endl;
    cout << "Prefix time: "
        << prefixDuration << " microseconds" << endl;
    Time postfixTime;
    auto startPostfix = chrono::high_resolution_clock::now();
    for (int i = 0; i < N; i++) {
       checksum = checksum + (postfixTime++).totalSeconds();
    }
    auto endPostfix = chrono::high_resolution_clock::now();
    long long postfixDuration =
       chrono::duration_cast<chrono::microseconds>(
          endPostfix - startPostfix).count();
    cout << "Postfix final seconds: "
        << postfixTime.totalSeconds() << endl;
    cout << "Postfix time: "
        << postfixDuration << " microseconds" << endl;
    cout << "Checksum: " << checksum << endl;
    return 0;
}
```





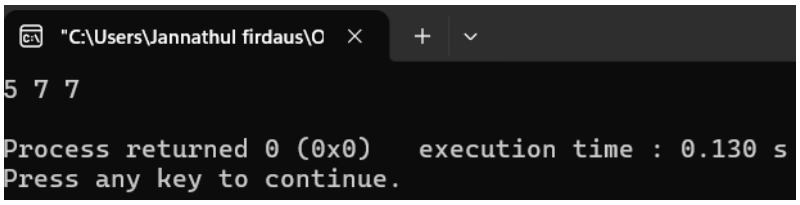
Prediction of Output

```
#include <iostream>
struct Counter {
    int v;
    Counter& operator++() {
        ++v;
        return *this;
    }
    Counter operator++(int) {
        Counter old = *this;
        ++v;
        return old;
    }
};
int main() {
    Counter c{5};
```

```
    Counter a = c++;
    Counter b = ++c;
    std::cout << a.v << ' '
              << b.v << ' '
              << c.v << '\n';
}
```

Predicted output:

5 7 7



Design and Analysis

A. Difference between prefix and postfix increment

Prefix ++i	**Postfix i++**
Increments the object first	Saves the old value first
Returns a reference to the updated object	Returns a copy of the old object
Uses no old-value temporary	Creates a temporary copy
Generally preferred for class types	Used when the old value is required

The compiler distinguishes the two overloads using the dummy int parameter:

```
Time& operator++();      // Prefix
Time operator++(int);    // Postfix
```

The dummy parameter is not used in the operation. It only distinguishes postfix increment from prefix increment.

B. Why does postfix increment return a copy?

Postfix increment must preserve the original value before incrementing the object. Therefore, it creates a temporary copy and returns that old value.

```
Time operator++(int) {
    Time old = *this;
    ++seconds;
```

```
    return old;
}
```

This may involve additional copying and temporary-object construction. Prefix increment does not need to preserve the old value, so it can return a reference to the current object.

C. Why are there two versions of operator[]**?**

The Playlist class provides two versions:

string& operator[](int index);

This non-const version allows the caller to read and modify a track.

const string& operator[](int index) const;

This const version allows read-only access when the Playlist object is const.

Both versions perform the same range check but provide different access permissions.

D. What happens when an index is out of range?

The valid index range is:

0 leq index < size

If the index is negative or equal to or greater than the playlist size, the function throws std::out_of_range.

This prevents invalid memory access and allows the calling program to handle the error using a try-catch block.

Experimental Investigation

A. Prefix and postfix values

For the expression:

Time t(0, 0, 5);

Time a = t++;

Time b = ++t;

The results are:

Variable	**Value**
Initial t	00:00:05
a after t++	00:00:05
b after ++t	00:00:07

Final t	00:00:07

B. Minute and hour rollover

The Time class stores time as total seconds rather than separately updating hours, minutes and seconds.

59 + 1 = 60 seconds

Therefore:

Before increment	After increment
00:00:59	00:01:00
00:59:59	01:00:00

The conversion from total seconds automatically handles minute and hour rollover.

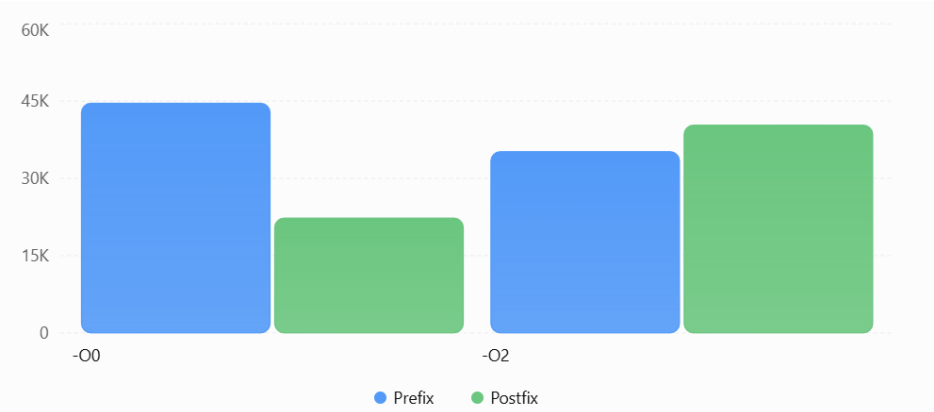
C. Performance measurement

The program performs 10,000,000 prefix and postfix increments under two compiler optimization settings.

Optimization	Prefix time	Postfix time
-O0	44,575 µs	22,272 µs
-O2	35,174 µs	40,326 µs

Increment performance comparison

Measured execution time in microseconds; lower is faster.



Alternative Concept and Complexity Analysis

A. Using at(i) **instead of overloaded** operator[]

The vector at() function performs bounds checking and throws std::out_of_range when the index is invalid.

Example:

```
vector<string> tracks = {
    "Song A", "Song B", "Song C"
};
try {
    cout << tracks.at(1);
}
catch (const out_of_range& e) {
    cout << "Invalid index";
}
```

B. Using unchecked operator[]

An unchecked implementation directly accesses the element:

```
string& operator[](int index) {
    return tracks[index];
}
```

It does not perform an explicit range check. Invalid indexes can result in undefined behaviour.

C. Complexity comparison

Access method	Time complexity	Range checking
Checked operator[]	O(1)	Yes
vector::at(i)	O(1)	Yes
Unchecked operator[]	O(1)	No

All three methods have constant-time access. Checked access performs an additional comparison to validate the index. at() also provides exception-based error handling.

D. Measuring the cost of range checking

A performance comparison can be performed using 10,000,000 accesses:

- Checked access: performs an index validation before accessing the element.
- Unchecked access: directly accesses the element.
- The execution times can be compared using chrono.

The extra check has constant-time complexity, although its actual runtime cost depends on compiler optimization and hardware.

Why Do Programmers Prefer ++i to i++ for Class Types?

Many programmers prefer prefix increment because it directly modifies the object and returns a reference to the updated object.

Postfix increment must preserve the previous value and return a copy. For class types, this can require additional temporary-object construction and copying.

Therefore, when the old value is not needed, prefix increment is generally the more efficient and recommended choice.

Q6 – The Rules of Operator Overloading

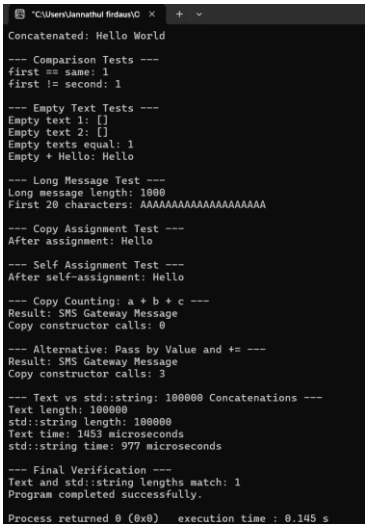
Aim

To implement a custom Text class using dynamic memory allocation and demonstrate deep copying, assignment, concatenation, comparison, printing, and memory management.

Implementation

The Text class includes:

- **Constructor:** Allocates memory and initializes text.
- **Copy constructor:** Creates an independent deep copy.
- **Assignment (=):** Copies text safely and prevents self-assignment.
- **Destructor:** Releases dynamically allocated memory using delete[].
- **Addition (+):** Concatenates two text objects.
- **Comparison (==, !=):** Compares text contents.
- **Stream insertion (<<):** Prints text.
- **Addition assignment (+=):** Appends text efficiently.



Design and Analysis

A. Operators that cannot be overloaded

The following operators cannot be overloaded in C++:

Operator	Name	Reason
::	Scope resolution	Determines scope at compile time.
.	Member access	Directly accesses class or object members.
.*	Pointer-to-member access	Has built-in member-pointer semantics.
?:	Conditional operator	Controls conditional expression evaluation.
sizeof	Size operator	Determines object or type size at compile time.

B. Rules of operator overloading

1. **Precedence cannot change:** An overloaded operator follows the precedence of its original operator.
2. **Associativity cannot change:** The original associativity rules remain unchanged.
3. **Arity cannot change:** The number of operands required by an operator cannot be modified.
4. **User-defined operand requirement:** At least one operand must be a user-defined type.
5. **Existing operators only:** C++ does not allow creating completely new operator symbols.

C. Why must operator= be a member function?

The assignment operator must be overloaded as a non-static member function because C++ requires the left operand of an overloaded assignment operator to be the object receiving the assignment.

For example:

Text first, second;

first = second;

Here, first is the left operand and receives the assigned data. The member function can directly access its private members and manage its dynamically allocated memory.

D. When should an overloaded operator be a friend?

A friend function is useful when an operator needs access to private members but should not be a member of the class.

For example, the stream insertion operator:

friend ostream& operator<<(ostream& out, const Text& text);

The left operand is ostream, not Text. Therefore, it is implemented as a non-member friend function to allow expressions such as:

cout << text;

The concatenation operator + can also be implemented as a friend function to provide symmetrical operand handling.

Experimental Investigation

A. Copy counting for a + b + c

Expression:

Text result = a + b + c;

The expression is evaluated from left to right in terms of grouping:

Text result = (a + b) + c;

The observed output was:

Result: SMS Gateway Message

Copy constructor calls: 0

The copy constructor was not called because the compiler could construct the intermediate and final result objects directly using copy elision.

B. Alternative implementation using pass-by-value and +=

Alternative function:

```
Text concatByValue(Text a, const Text& b) {
   a += b;
   return a;
}
```

Actual output:

Result: SMS Gateway Message

Copy constructor calls: 3

This implementation produced three copy constructor calls in the experiment. Passing arguments by value creates additional copies, whereas passing by reference avoids unnecessary copying.

C. Self-assignment test

Statement:

assigned = assigned;

Output:

After self-assignment: Hello

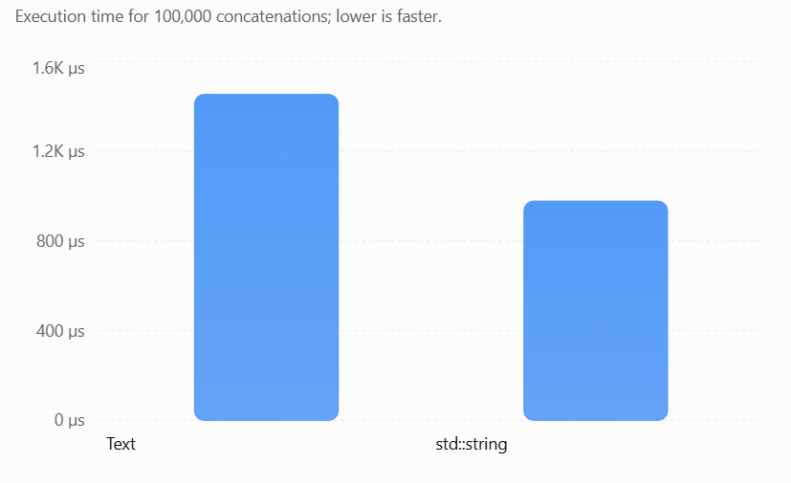
The self-assignment check prevents the object from unnecessarily copying itself and protects its dynamically allocated memory.

D. Performance comparison with std::string

Both implementations performed 100,000 concatenations.

Measurement	Custom Text	std::string
Concatenations	100,000	100,000
Final length	100,000	100,000
Execution time	1453 µs	977 µs

Concatenation Performance



7. Alternative Concept and Complexity Analysis

A. Using std::string

The same concatenation can be implemented using:

string a = "SMS ";

string b = "Gateway ";

string c = "Message";

string result = a + b + c;

std::string automatically manages memory allocation, copying, and resizing. It reduces the possibility of memory-management errors and is generally preferred for normal string-handling applications.

B. Complexity of a + b + c

Let:

- n_1 = length of string a
- n_2= length of string b
- n_3 = length of string c

For the expression:

a + b + c

The first operation creates a + b, copying:

n_1+n_2

characters.

The second operation appends c to the intermediate result, copying:

n_1+n_2+n_3

characters.

Therefore, the total number of character copies is:

(n_1+n_2)+(n_1+n_2+n_3)

2n_1+2n_2+n_3

The time complexity is:

O(n_1+n_2+n_3)

C. Temporary space analysis

Implementation	**Temporary space**
Custom Text with +	Intermediate string of length \(n_1+n_2\), followed by final string of length \(n_1+n_2+n_3\).
std::string with +	Similar intermediate and final string storage, with implementation-managed allocation.
Pass-by-value with +=	Additional copied parameter objects and intermediate storage, depending on copy elision and move optimization.

The peak additional storage for concatenation is proportional to:

O(n_1+n_2+n_3)

Using += repeatedly on a single result object can reduce temporary allocations. The custom implementation also uses capacity growth to reduce repeated reallocations.

Q7 – Design a Complex-Number Class from First Principles

Stage 1 – Interpret the Problem

Objective: Design a C++ class named Complex to represent and perform arithmetic operations on complex numbers. The class will also be used to calculate impedance in AC electrical circuits.

A complex number is represented as:

$$z = a + jb$$

where:

- a = real part
- b = imaginary part
- $j = \sqrt{-1}$

Class requirements:

1. Store real and imaginary parts as private data members.
2. Provide default, single-argument and two-argument constructors.
3. Overload arithmetic operators +, -, * and /.
4. Overload the equality operator ==.
5. Implement a magnitude() function.
6. Overload the stream insertion operator <<.
7. Handle division by zero safely.
8. Apply the class to series and parallel AC circuit impedance calculations.

Stage 2 – Design the Solution

Class structure

The Complex class contains two private data members:

double real;

double imag;

Constructors

Constructor	**Purpose**
Complex()	Initializes both parts to zero

Complex(double r)	Initializes the real part only
Complex(double r, double i)	Initializes both real and imaginary parts

Operator overloading

Operator / Function	**Purpose**
+	Adds two complex numbers
-	Subtracts two complex numbers
*	Multiplies two complex numbers
/	Divides two complex numbers
==	Compares two complex numbers
<<	Displays a complex number
magnitude()	Calculates the magnitude

Mathematical formulas

For two complex numbers:

$$A = a + jb, \quad B = c + jd$$

$$A + B = (a + c) + j(b + d)$$

$$A - B = (a - c) + j(b - d)$$

$$AB = (ac - bd) + j(ad + bc)$$

$$\frac{A}{B} = \frac{(ac + bd) + j(bc - ad)}{c^2 + d^2}$$

$$|A| = \sqrt{a^2 + b^2}$$

AC circuit impedance

$$Z = R + jX_L - jX_C$$

$$Z = \frac{Z_1 Z_2}{Z_1 + Z_2}$$

Code:

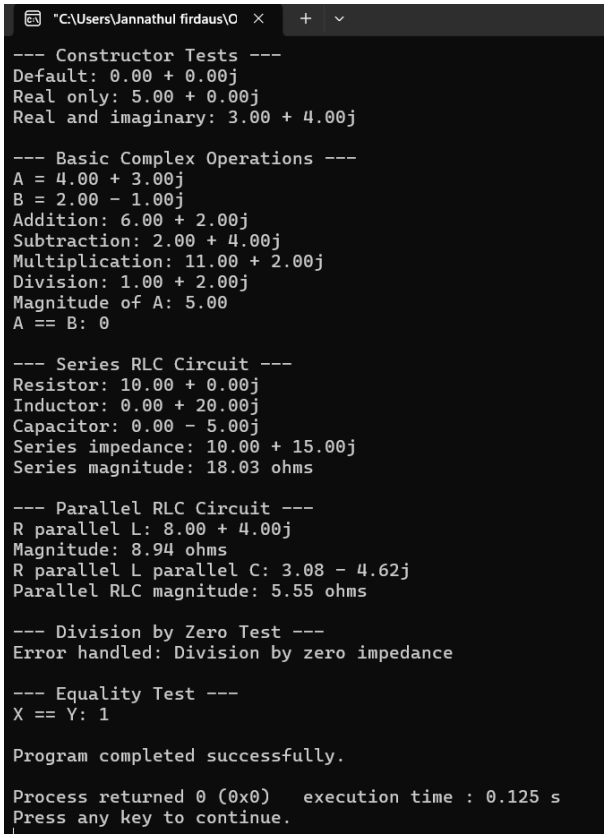
```
#include <iostream>
#include <cmath>
#include <stdexcept>
#include <iomanip>
using namespace std;
class Complex {
private:
    double real;
    double imag;
public:
    // Default constructor
    Complex() {
        real = 0.0;
        imag = 0.0;
    }
    // Constructor with real part only
    Complex(double r) {
        real = r;
        imag = 0.0;
    }
    // Constructor with real and imaginary parts
    Complex(double r, double i) {
        real = r;
        imag = i;
    }
    // Addition
    Complex operator+(const Complex& other) const {
        return Complex(real + other.real,
                       imag + other.imag);
```

```
}
// Subtraction
Complex operator-(const Complex& other) const {
    return Complex(real - other.real,
                   imag - other.imag);
}
// Multiplication
Complex operator*(const Complex& other) const {
    return Complex(
        real * other.real - imag * other.imag,
        real * other.imag + imag * other.real
    );
}
// Division
Complex operator/(const Complex& other) const {
    double denominator =
        other.real * other.real +
        other.imag * other.imag;
    if (denominator == 0.0) {
        throw runtime_error("Division by zero impedance");
    }
    return Complex(
        (real * other.real + imag * other.imag) / denominator,
        (imag * other.real - real * other.imag) / denominator
    );
}
// Equality
bool operator==(const Complex& other) const {
    const double EPSILON = 1e-9;
```

```
        return fabs(real - other.real) < EPSILON &&
               fabs(imag - other.imag) < EPSILON;
    }
    // Magnitude
    double magnitude() const {
        return sqrt(real * real + imag * imag);
    }
    // Stream insertion
    friend ostream& operator<<(ostream& out,
                               const Complex& value) {
        out << fixed << setprecision(2)
            << value.real;
        if (value.imag >= 0)
            out << " + " << value.imag << "j";
        else
            out << " - " << fabs(value.imag) << "j";
        return out;
    }
};
int main() {
    cout << fixed << setprecision(2);
    cout << "--- Constructor Tests ---" << endl;
    Complex z1;
    Complex z2(5.0);
    Complex z3(3.0, 4.0);
    cout << "Default: " << z1 << endl;
    cout << "Real only: " << z2 << endl;
    cout << "Real and imaginary: " << z3 << endl;
```

```
cout << "\n--- Basic Complex Operations ---" << endl;
Complex a(4.0, 3.0);
Complex b(2.0, -1.0);
cout << "A = " << a << endl;
cout << "B = " << b << endl;
cout << "Addition: " << a + b << endl;
cout << "Subtraction: " << a - b << endl;
cout << "Multiplication: " << a * b << endl;
cout << "Division: " << a / b << endl;
cout << "Magnitude of A: " << a.magnitude() << endl;
cout << "A == B: " << (a == b) << endl;
cout << "\n--- Series RLC Circuit ---" << endl;
// R = 10 ohms
// Inductive reactance XL = 20 ohms
// Capacitive reactance XC = 5 ohms
Complex R(10.0, 0.0);
Complex L(0.0, 20.0);
Complex C(0.0, -5.0);
Complex series = R + L + C;
cout << "Resistor: " << R << endl;
cout << "Inductor: " << L << endl;
cout << "Capacitor: " << C << endl;
cout << "Series impedance: " << series << endl;
cout << "Series magnitude: " << series.magnitude()
    << " ohms" << endl;
cout << "\n--- Parallel RLC Circuit ---" << endl;
// Parallel combination of the same impedances
Complex parallel = (R * L) / (R + L);
cout << "R parallel L: " << parallel << endl;
```

```
    cout << "Magnitude: " << parallel.magnitude()
         << " ohms" << endl;
    // Parallel combination of R, L and C
    Complex parallelRLC =
        (R * L * C) /
        (R * L + L * C + C * R);
    cout << "R parallel L parallel C: "
         << parallelRLC << endl;
    cout << "Parallel RLC magnitude: "
         << parallelRLC.magnitude() << " ohms" << endl;
    cout << "\n--- Division by Zero Test ---" << endl;
    try {
        Complex zero(0.0, 0.0);
        Complex result = a / zero;
        cout << result << endl;
    }
    catch (const runtime_error& error) {
        cout << "Error handled: " << error.what() << endl;
    }
    cout << "\n--- Equality Test ---" << endl;
    Complex x(3.0, 4.0);
    Complex y(3.0, 4.0);
    cout << "X == Y: " << (x == y) << endl;
    cout << "\nProgram completed successfully." << endl;
    return 0;
}
```



Hand calculation and verification

$$R = 10, \quad X_L = 20, \quad X_C = 5$$

$$Z = 10 + j20 - j5$$

$$\boxed{Z = 10 + j15}$$

$$|Z| = \sqrt{10^2 + 15^2} = \sqrt{325} = 18.03\ \Omega$$

$$Z = \frac{10(j20)}{10 + j20}$$

$$\boxed{Z = 8 + j4}$$

$$|Z| = \sqrt{8^2 + 4^2} = 8.94\ \Omega$$

$$Z = \frac{RLC}{RL + LC + CR}$$

$$|Z| \approx 5.55\ \Omega$$

The calculated values match the program output.

Verification results

- All three constructors work correctly.
- Arithmetic operators produce the expected results.
- The magnitude function calculates the correct value.
- Equality comparison works correctly.
- Stream insertion displays complex numbers in the required format.
- Division by zero is handled using an exception.
- Series and parallel circuit calculations produce the expected impedance values.

Stage 4 – Defend the Design

Comparison with the standard library

C++ provides the built-in std::complex<double> class through the <complex> header.

Example:

```
#include <iostream>
#include <complex>
using namespace std;
int main() {
    complex<double> A(4, 3);
    complex<double> B(2, -1);
    cout << "Addition: " << A + B << endl;
    cout << "Subtraction: " << A - B << endl;
    cout << "Multiplication: " << A * B << endl;
    cout << "Division: " << A / B << endl;
    cout << "Magnitude: " << abs(A) << endl;
    return 0;
}
```

Feature	Custom Complex Class	std::complex
Constructors	User-defined	Built-in
Arithmetic	Operator overloading	Built-in
Magnitude	Custom function	abs()

Equality	Custom implementation	Built-in
Output	Custom formatting	Standard formatting
Error handling	Custom exception	No automatic matching custom exception

Why operator overloading is useful

Operator overloading makes complex-number calculations easier to read and understand. For example:

Complex result = A + B;

This is more natural than calling a separate addition function.

Why the stream operator is a friend

The << operator is overloaded as a friend function because the output stream is the left operand. This allows the function to access the private real and imaginary values.

Design limitation

The division operator checks whether the denominator is exactly zero. For floating-point calculations, extremely small denominator values can also cause numerical instability. A more robust implementation could use a tolerance-based check.

Alternative Concept and Complexity Analysis

Alternative approach: Instead of creating a custom class, the C++ standard library's std::complex<double> can be used. It provides ready-made arithmetic operators and magnitude calculations.

Time and space complexity

Operation	**Time Complexity**	**Extra Space**
Addition	O(1)	O(1)
Subtraction	O(1)	O(1)
Multiplication	O(1)	O(1)
Division	O(1)	O(1)
Equality	O(1)	O(1)
Magnitude	O(1)	O(1)

Engineering justification

The custom class provides control over the implementation, output format and error handling. It is useful for understanding operator overloading and applying complex arithmetic to

electrical engineering calculations. For larger scientific applications, std::complex<double> is generally preferable because it is standardized and widely tested.